

IDENTITY GOVERNANCE VALIDATION ENGINE

Technical Reference | Architecture & Engineering

Version 2.0 · March 2026

Platform: Microsoft Azure / Entra ID

Classification: Internal — IAM Engineering

1. Purpose and Scope

1.1 What Problem This Solves

Enterprise IAM environments accumulate risk silently. Accounts persist after employees leave. Contractors end up in manager-tier groups. Azure Owner roles get assigned directly to users instead of through groups. Privileged accounts go without MFA. None of these surface in routine Entra ID reports because the platform records state — it does not evaluate it.

The Identity Governance Validation Engine is a PowerShell-based compliance and risk assessment system purpose-built for Microsoft Azure and Entra ID environments. It reads live identity and RBAC state from the Microsoft Graph and Azure ARM APIs, evaluates every identity in the tenant against a configurable rule set, and produces a structured output that answers two questions deterministically: is this identity compliant, and how much risk does it represent.

The engine was designed to solve four specific problems that static audits cannot:

- IAM drift — identities accumulate permissions over time that were never explicitly granted for their current role
- Lifecycle misalignment — accounts remain active and permissioned after employment ends or changes
- RBAC sprawl — direct Azure role assignments bypass group-based access controls, creating untracked privilege
- Hygiene gaps — inactive accounts and missing MFA on privileged identities create persistent exposure that goes undetected between audit cycles

1.2 Non-Goals

The engine intentionally does not:

- Modify any identity, group membership, or role assignment — it reads state only
- Replace Entra ID Protection or Microsoft Defender for Identity — those detect attack patterns; this evaluates configuration compliance against a defined entitlement model
- Process real-time events or signals — it produces point-in-time snapshots on demand
- Evaluate Entra ID directory roles or PIM eligibility — scope is Azure RBAC and Entra group memberships only
- Evaluate Conditional Access policy compliance or device state
- Operate on guest or B2B accounts — external identities are excluded from evaluation

1.3 Scope Boundary

In Scope	Out of Scope
Entra ID member users (cloud and hybrid-synced)	Guest / B2B / external users
Azure RBAC role assignments across all subscriptions	Entra ID directory roles and PIM assignments
Security group memberships vs entitlement model	Conditional Access policies
MFA registration state on privileged accounts	Device compliance state
Employment status alignment via EmployeeType / EmployeeId	OAuth application permission scopes
Sign-in activity (requires AuditLog.Read.All)	Management group RBAC (detected but not deep-evaluated)

2. Architecture

2.1 Design Principles

The engine was built around three principles that shaped every structural decision:

- Collectors are fact-only. No rule logic, no filtering, no interpretation. A collector's only job is to call an API, normalise the response into a consistent object shape, and return it. This means any bug in a collector is a data bug, not a logic bug — the two failure modes are cleanly separated.
- Processors are rule-only. No API calls, no state mutation. A processor receives a snapshot and a rules document and emits findings. It cannot reach into another layer. This makes rules testable in isolation with mock snapshots.
- Classification is separate from evaluation. The RiskClassifier receives a flat list of findings and produces risk states. It has no knowledge of what any rule means — it only knows weights, severity labels, and blocking flags. This means the scoring model can change without touching any rule logic.

2.2 Layered Architecture

Layer	Files	Responsibility	Can Call
Orchestration	ValidationEngine.ps1	Wires all layers in order. Owns enrichment map (identity context → report). Sole file permitted to call across layer boundaries.	All layers
Collection	IdentityCollector.ps1 RbacCollector.ps1	Reads live state from Graph and ARM. Returns normalised snapshots. Contains zero rule logic.	Microsoft Graph, Azure ARM only
Schema	FindingSchema.ps1	Defines the single finding object shape. All processors must use New-ComplianceFinding — no processor builds findings manually. Also hosts Invoke-SuppressionPass.	None (pure constructor)
Rule Evaluation	IDRuleProcessor.ps1 RbacRuleProcessor.ps1 CorrelationRuleProcessor.ps1	Applies Rules.json against collected snapshots. Emits findings via FindingSchema. Calls Invoke-SuppressionPass at return.	FindingSchema only
Classification	RiskClassifier.ps1	Aggregates findings into per-entity risk states and tenant summary. Contains no Azure API calls and no rule logic.	None
Reporting	ReportGenerator.ps1	Writes console output, CSV, JSON, and drift state file.	None

2.3 Execution Pipeline

Each run follows a fixed five-step sequence regardless of mode. Steps are skipped only in PreProvision mode where subscription scanning is not needed.

Step	Action	Output Object	Skipped In
1	Identity collection — Graph API call, normalisation, privilege flag resolution	IdentitySnapshot (Users, Groups, MembershipMap)	Never
2	RBAC collection — ARM enumeration of all subscriptions and role assignments	RbacSnapshot (RoleAssignments, SubscriptionsEnumerated)	PreProvision

Step	Action	Output Object	Skipped In
3	Rule evaluation — IDRuleProcessor, RbacRuleProcessor, CorrelationRuleProcessor run in sequence; suppression passes run at each orchestrator return	List[PSCustomObject] findings (flat)	RBAC + Correlation skipped in PreProvision
4	Risk classification — findings grouped by entity, dual-model state computed per entity, tenant summary produced	ClassificationResult (EntityRiskStates, Summary)	Never
5	Report generation — console output, optional CSV, optional JSON, optional drift state write	Files on disk, return value to caller	Never

2.4 Why the Architecture Is Subscription-Agnostic

An early design question was whether to build the engine around a specific subscription or set of subscriptions. The decision to make RBAC collection subscription-agnostic — scanning every subscription the service account can see — was deliberate and driven by three problems:

- Organisations routinely have RBAC assignments in subscriptions that IAM teams do not know exist. Scoping to a known list would produce false confidence — the engine would report clean where it had simply not looked.
- The correlation rules (CORR-001 through CORR-005) require the full RBAC picture to be meaningful. A privileged identity holding Owner in subscription B is equally dangerous whether subscription B was in the filter or not.
- The SubscriptionFilter parameter is provided for cases where scope restriction is a genuine operational requirement (e.g. evaluating a specific business unit). It is opt-in, not the default — the default is always complete visibility.

The trade-off is performance: scanning N subscriptions multiplies the ARM API call volume by N. For large organisations with dozens of subscriptions, this is a real consideration. The current architecture is sequential per subscription. Parallelisation is a future enhancement — see Section 9.

3. Scan Modes

The engine supports three modes. Mode selection determines which collectors run, which rule categories are evaluated, and what the output contract is. Mode behaviour is deterministic — the same input always produces the same output for a given mode.

3.1 FullScan

Property	Value
Trigger	Scheduled (daily/weekly) or on-demand compliance review
Collectors	IdentityCollector + RbacCollector (both run)
Rule categories	All: Identity, Access, Architecture, Hygiene, RBAC, Correlation
Entities evaluated	All users in the tenant + any Group/SP principals appearing in RBAC findings
Output	Console + optional CSV + optional JSON + optional drift state
Performance	Heaviest mode — Graph enumerate-all + ARM per-subscription. Scales with tenant size.
Risk tolerance	Full signal. All 26 rules active. Hygiene findings separated from compliance gate.

```
.\ValidationEngine.ps1 -Mode FullScan -ExportCsv -ExportJson -StoreDriftState -ResolveMfa
```

3.2 PreProvision

Property	Value
Trigger	JML workflow — called before any group assignment or resource access is granted to a new or moving identity
Collectors	IdentityCollector only (single user, filtered by TargetUserId). RBAC collection skipped.
Rule categories	Identity + Access only. RBAC and Correlation skipped (no subscription data).
Entities evaluated	Single user specified by -TargetUserId (Azure AD Object ID)
Output	Decision: Pass Fail. BlockingCount. Reasons[] (human-readable strings). Structured PSCustomObject returned to caller.
Performance	Fastest mode. Single Graph user fetch + group membership resolution. No ARM calls.
Risk tolerance	Zero tolerance for blocking findings. Only blocking rules matter — non-blocking findings are collected but do not affect Decision.

```
.\ValidationEngine.ps1 -Mode PreProvision -TargetUserId "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
```

CONTRACT PreProvision returns a PSCustomObject with: Decision (Pass|Fail), BlockingCount (int), Reasons (string[]), Findings (PSCustomObject[]), EvaluatedAt (ISO 8601). The JML caller must check Decision — it is the only field guaranteed stable across engine versions.

3.3 DriftOnly

Property	Value
Trigger	Lightweight scheduled scan between FullScan runs. Detects structural changes quickly.
Collectors	IdentityCollector + RbacCollector (both run)
Rule categories	Architecture, RBAC, Access, Correlation only. Identity and Hygiene rules skipped.
Entities evaluated	Only entities with drift-category findings (entities with no structural drift are omitted from output)
Output	Console + optional JSON. Summary shows TotalEntitiesWithDrift.
Performance	Lighter than FullScan — fewer rule evaluations, smaller output. Same API call volume.
Risk tolerance	Structural drift only. Hygiene debt and employment misalignment are not evaluated.

```
.\ValidationEngine.ps1 -Mode DriftOnly -ExportJson
```

4. Data Model

4.1 Identity Snapshot Schema

The IdentitySnapshot is the canonical representation of Entra ID state. It is produced once per run and shared across all rule processors.

Field	Type	Source	Notes
UserId	string (GUID)	Graph id	Azure AD Object ID — primary key
DisplayName	string	Graph displayName	Trimmed — Graph can return trailing whitespace
UserPrincipalName	string	Graph userPrincipalName	Validated by IDENT-002
AccountEnabled	bool	Graph accountEnabled	Disabled accounts still appear in snapshot
EmployeeType	string	Graph employeeType	HR-sourced via Entra HR provisioning. Raw value.
EmployeeId	string	Graph employeeId	Presence/absence used in employment status resolution
LastSignInDateTime	DateTime?	Graph signInActivity	Null if AuditLog.Read.All scope missing or P1/P2 not licensed
LastPasswordChangeDateTime	DateTime?	Graph lastPasswordChangeDateTime	Used by HYG-003
OnPremisesSynced	bool	Graph onPremisesSyncEnabled	Informational — hybrid sync flag
MfaRegistered	bool?	Get-MgUserAuthenticationMethod	Null unless -ResolveMfa switch used. Optional due to per-user API cost.
EmploymentStatus	string	Derived by Resolve-EmploymentStatus	Active Terminated Offboarded Unknown Guest
IsPrivileged	bool	Derived by Set-UserPrivilegeFlag	True if member of any privileged-tier group

4.2 RBAC Snapshot Schema

Field	Type	Notes
PrincipalId	string (GUID)	Azure AD Object ID of the role assignment subject
PrincipalName	string	DisplayName from ARM. Used when principal is not in identity snapshot (SP, Group).
PrincipalType	string	User Group ServicePrincipal
RoleDefinitionName	string	Human-readable role name (e.g. Owner, Contributor)
Scope	string	Full ARM scope path
ScopeType	string	Subscription ResourceGroup Resource ManagementGroup Unknown
SubscriptionId	string	Extracted from Scope path
AssignmentType	string	Direct (User) Inherited (Group/SP)

4.3 Finding Schema

Every finding produced by every rule processor must be constructed via `New-ComplianceFinding`. No processor builds findings manually. This contract is enforced by the function signature.

Field	Type	Values / Notes
EntityId	string (GUID)	Azure AD Object ID of the evaluated entity
EntityType	string	User Group ServicePrincipal App Subscription Device Policy
RuleId	string	e.g. IDENT-001, HYG-004. Stable identifier — safe for downstream automation.
Category	string	Identity Access Architecture Hygiene RBAC Correlation
Severity	string	Low Medium High Critical
Weight	int (0–100)	Numeric risk contribution. Bounded by schema — prevents scoring manipulation.
Blocking	bool	True = hard block in PreProvision. True = Blocking ComplianceStatus in FullScan.
Details	string	Human-readable finding description. Surfaced in reports and Reasons[[]].
Timestamp	string (ISO 8601)	UTC. Format suitable for SIEM ingestion.
Mode	string	FullScan DeltaScan Simulation TargetedScan

4.4 Risk Scoring Model

Dual-Model Classification

The engine produces two independent outputs per entity. They serve different purposes and must not be conflated.

Output	Purpose	Values	Gate Logic
ComplianceStatus	JML workflow decisions and compliance reporting	Compliant NonCompliant Blocking	Driven by compliance findings only. Non-blocking Hygiene findings (HYG-001/002/003) are excluded from this gate. Blocking Hygiene findings (HYG-004: no MFA on privileged) remain included.
RiskLevel	Security triage and dashboarding	None Low Medium High Critical	Driven by full finding set including hygiene. Informational only — does not gate workflows.
RiskScore	Trend analysis and entity ranking	0.0 – 100.0 (normalised)	Raw weight sum normalised against fixed theoretical maximum of 900. See formula below.
RiskDensity	Distinguish concentrated vs broad risk	RiskScore ÷ total violation count	High density = few severe findings. Low density = many minor findings across a large surface.

Scoring Formula

`RiskScoreRaw` is the sum of `Weight` values across all findings for an entity (compliance + hygiene). `RiskScore` normalises this to 0–100:

```
RiskScore = Min( Round( (RiskScoreRaw / 900) × 100, 1 ), 100.0 )
```

```
Theoretical maximum (900) = worst-case single entity stacking all critical rules:
  HYG-004 (100) + RBAC-003 (100) + CORR-001 (100) + ACCESS-002 (100)
+ ENT-002 (100) + RBAC-001 (80) + CORR-003 (85) + ACCESS-003 (75)
+ RBAC-002 (75) + ENT-001 (70) = 885
```


Using 900 as cap ensures no realistic entity reaches exactly 100.0 under normal conditions.
A fixed constant keeps scores comparable across tenants and over time.
Tenant-relative normalisation was rejected – it would compress clean tenants when one severely non-compliant entity exists.

RecommendedAction Mapping

ComplianceStatus	Hygiene Findings Present	RecommendedAction
Compliant	No	None
Compliant	Yes	HygieneReview
NonCompliant	No critical severity	Alert
NonCompliant	Critical severity present	Review
Blocking	Any	Escalate

5. Rule Framework

5.1 Rules.json Structure

All rule definitions live in Rules.json. The rule engine is data-driven — processors read rule definitions at runtime and dispatch to the appropriate evaluator function based on the type field. Adding a new rule requires only a new entry in Rules.json and a corresponding evaluator function in the relevant processor.

```
{
  "version": "2.0.0",
  "rules": [ ... ],           // 26 rule definitions
  "entitlementModel": {
    "_privilegedTiers": ["Manager", "Manager/Staff"],
    "groups": { ... }         // 15 groups with tier and allowedEmployment
  }
}
```

5.2 Suppression Chain

When two rules would fire for the same finding on the same entity, the more specific rule suppresses the less specific one. This prevents severity inflation from multiple rules describing one security fact. Suppression is declared in Rules.json (not hardcoded) and applied by Invoke-SuppressionPass at the return of every orchestrator.

Suppressor	Suppresses	Reason
HYG-002 (180-day inactivity)	HYG-001 (90-day inactivity)	Both fire on the same null/stale sign-in. 180-day is more severe and fully describes the condition.
RBAC-003 (forbidden role)	RBAC-001 (direct assignment)	Forbidden role implies direct assignment. Reporting both is redundant.
RBAC-003 (forbidden role)	RBAC-002 (broad scope)	Scope-aware suppression: RBAC-002 suppressed only when RBAC-003 fires for same entity+scope+role. Handled by dedicated scope-key pass in RbacRuleProcessor.
ENT-002 (privileged tier violation)	ACCESS-001 (job title pattern match)	ENT-002 uses the authoritative entitlement model. ACCESS-001 is a pattern-match fallback. Model-based result is always more precise.
CORR-001 (priv group + Owner at sub)	CORR-004 (priv identity + direct assignment)	CORR-001 is a strict superset of CORR-004. When both would fire, CORR-001 fully describes the risk.

5.3 Complete Rule Reference

All 26 rules. Weight is the numeric risk contribution (0–100). Blocking=Yes means the rule causes ComplianceStatus=Blocking in FullScan and Decision=Fail in PreProvision.

Identity Category

Rule ID	Severity	W	Blocking	Description	Key Dependency
IDENT-001	Critical	100	Yes	Disabled account remains in one or more privileged groups	Group membership + AccountEnabled flag

Rule ID	Severity	W	Blocking	Description	Key Dependency
IDENT-002	High	70	Yes	Account has no UPN or malformed UPN (regex: <code>^[^@]+@[^@]+\.[^@]+\$</code>)	UserPrincipalName field
IDENT-003	Medium	40	No	Employment status not Active or unresolvable (Unknown). Covers missing HR record and null EmployeeType.	EmployeeType + EmployeeId fields
JOIN-001	Critical	100	Yes	Joiner: employment record not found — block provisioning. Active within 30-day joiner window.	EmploymentStatus = Active
JOIN-002	High	80	Yes	Joiner: job title maps to a restricted privileged group — block provisioning	JobTitle pattern + group tier

Access Category

Rule ID	Severity	W	Blocking	Description	Key Dependency
ACCESS-001	High	80	Yes	Intern or contractor in a Manager-tier privileged group. Superseded by ENT-002 when entitlement model coverage exists.	JobTitle pattern + group name pattern
ACCESS-002	Critical	100	Yes	Terminated or offboarded account retains active group memberships	EmploymentStatus (Terminated/Offboarded/Left/Resigned) + any group membership
ACCESS-003	High	75	No	Account is in both privileged-tier and standard-tier groups simultaneously — mixed membership	IsPrivileged flag + non-privileged group membership
ENT-001	High	70	No	Employment type not permitted for assigned group per entitlement model — flagged for review	EmployeeType vs group allowedEmployment in Rules.json
ENT-002	Critical	100	Yes	Employment type not permitted for a privileged-tier group — block provisioning. Suppresses ACCESS-001.	EmployeeType vs privileged group allowedEmployment
ENT-003	Low	15	No	User is in a group not defined in the entitlement model. Coverage gap, not a compliance failure.	Group not in Rules.json entitlementModel.groups

Architecture Category

Rule ID	Severity	W	Blocking	Description	Key Dependency
ARCH-001	Critical	100	Yes	Tier 0 asset managed by a non-Tier 0 account (patterns: Tier0, DomainAdmins, EnterpriseAdmins)	Group name pattern match
ARCH-002	High	70	No	Admin account used for day-to-day operations — no account separation (patterns: adm_, admin_, _admin, svc_)	UPN prefix/suffix pattern match
ARCH-003	High	65	No	Service account in interactive user groups (svc_ pattern in groups like AllUsers, Staff)	UPN pattern + group name pattern

Hygiene Category

Hygiene rules **do not gate ComplianceStatus** except HYG-004. Non-blocking hygiene findings (HYG-001/002/003) contribute to RiskScore but leave ComplianceStatus=Compliant. This separation prevents operational debt from inflating compliance statistics.

Rule ID	Severity	W	Blocking	Description	API Dependency
HYG-001	Medium	35	No	Account inactive >90 days. Superseded by HYG-002 when both fire. Hygiene only.	LastSignInDateTime — requires AuditLog.Read.All
HYG-002	High	65	No	Account inactive >180 days (stale). Suppresses HYG-001. Fires on null LastSignInDateTime when AuditLog scope is missing.	LastSignInDateTime — requires AuditLog.Read.All
HYG-003	Medium	40	No	Account password not changed in >365 days	LastPasswordChangeDateTime — Graph base scope
HYG-004	Critical	100	Yes	Privileged account has no MFA registered or enforced. Exception to hygiene non-blocking rule — MFA on privileged accounts is a hard security control.	UserAuthenticationMethod.Read.All + -ResolveMfa switch

RBAC Category

Rule ID	Severity	W	Blocking	Description	Suppression
RBAC-001	High	80	No	Direct Azure role assignment to a user — must be group-based. Suppressed by RBAC-003 when a forbidden role is also present.	Superseded by RBAC-003
RBAC-002	High	75	No	Privileged role (Owner/Contributor/UAA) at subscription scope. Scope-aware suppression: only suppressed by RBAC-003 when same entity+scope+role.	Superseded by RBAC-003 (scope-aware)
RBAC-003	Critical	100	Yes	Owner role directly assigned to a user at any scope. Suppresses RBAC-001 and RBAC-002 for the same entity. Primary RBAC blocking rule.	Suppresses RBAC-001, RBAC-002

Correlation Category

Correlation rules cross the identity and RBAC planes. They require both the identity snapshot and the RBAC snapshot and are evaluated only in FullScan and DriftOnly modes.

Rule ID	Severity	W	Blocking	Description	Suppression
CORR-001	Critical	100	Yes	Privileged-tier group member holds Owner or User Access Administrator at subscription scope. Most severe cross-plane finding. Suppresses CORR-004.	Suppresses CORR-004
CORR-002	Critical	100	Yes	Disabled account retains active Azure RBAC role assignments	None
CORR-003	High	85	Yes	Non-full-time worker (Intern/Contractor/Temporary) holds a	None

Rule ID	Severity	W	Blocking	Description	Suppression
				privileged Azure RBAC role (Owner/Contributor/UAA)	
CORR-004	High	75	No	Privileged-tier identity has a direct Azure role assignment. Subset of CORR-001 — fires only when CORR-001 does not.	Superseded by CORR-001
CORR-005	High	70	No	Account inactive >90 days but retains Azure RBAC role assignments	None

6. Data Quality and Failure Handling

The engine was designed to handle partial data gracefully. Missing scopes, expired sessions, and throttled APIs all occur in production. The behaviour in each case is documented here because silent failures in data collection produce misleading compliance results.

6.1 API Scope Requirements

Scope	Required For	Failure Mode if Missing
User.Read.All	All user enumeration	Engine aborts — no identity snapshot possible
Group.Read.All	Group enumeration and privilege classification	Engine aborts — no membership resolution possible
GroupMember.Read.All	Membership map (UserId → GroupId[])	Engine aborts — no privilege flags, no entitlement evaluation
UserAuthenticationMethod.Read.All	MFA state resolution (- ResolveMfa)	MfaRegistered stays null. HYG-004 will not fire. Privileged accounts appear MFA-compliant when they may not be.
AuditLog.Read.All	LastSignInDateTime population	LastSignInDateTime is null for all users. HYG-001 and HYG-002 fire tenant-wide (43/48 entities in observed run). Results are misleading until scope is added.
Az.Accounts + Az.Resources	RBAC snapshot (ARM)	RbacSnapshot is empty. All RBAC and Correlation rules produce zero findings. RBAC risk is invisible.

OBSERVED FAILURE In the initial production run, AuditLog.Read.All was missing from the Connect-MgGraph call. HYG-002 fired on all 43/48 non-compliant entities with "No sign-in recorded" — not because accounts were inactive, but because LastSignInDateTime was null. This produced a 0% compliant distribution that appeared to reflect real compliance debt but was primarily a data gap. The fix was adding AuditLog.Read.All to the Graph connection scope.

6.2 Session and Context Failures

Failure	Detection	Behaviour
Azure session expired mid-run	Get-AzContext returns null after Set-AzContext	RbacCollector throws explicitly: "Azure context is null — session may have expired." Run aborts with clear message rather than returning silently empty RBAC snapshot.
Subscription enumeration fails	Get-AzSubscription throws	Empty RbacSnapshot returned. Error logged to ErrorLogPath CSV if configured. RBAC/Correlation findings will be zero — report will note this.
Individual subscription RBAC query fails	Get-AzRoleAssignment throws per-subscription	Error logged. Other subscriptions continue. Partial RBAC snapshot returned — findings may be incomplete for the failed subscription.
Graph user enumeration fails	Get-MgUser throws	Empty user list returned. IdentitySnapshot.Users = 0. Engine writes warning and returns null for non-PreProvision modes.
Individual group membership fails	Get-MgGroupMember throws per group	Error logged to ErrorLogPath CSV. Other groups continue. Membership for failed group is absent from snapshot — affected users may appear with fewer group memberships than they actually have.

6.3 signInActivity License Detection

The `LastSignInDateTime` field from Graph `signInActivity` requires either Entra ID P1 or P2 licensing, or the `AuditLog.Read.All` scope. The collector probes for this capability before the full user enumeration:

```
# Capability probe — single user, signInActivity property
# If probe succeeds: signInActivity included for all users
# If probe fails (403 / not licensed): field excluded, LastSignInDateTime = null
$probe = Get-MgUser -Top 1 -Property "id,signInActivity" -ErrorAction Stop
```

This avoids failing the entire collection when the license or scope is absent. The trade-off is that HYG-001 and HYG-002 will fire on every user when the capability is unavailable — a known and documented false positive condition.

6.4 Logging Strategy

- Non-fatal collection errors (individual group membership failures, individual subscription failures) are written to an optional `ErrorLogPath` CSV with `Timestamp`, `Source`, and `Detail` fields.
- Fatal failures (no users, expired session) cause an explicit throw or `Write-Warning` and early return.
- All rule evaluation errors are caught per-rule and written to `Write-Debug` — a failing rule does not stop the run.
- No retry logic is currently implemented. Graph throttling (429 responses) would surface as a failed collection error.

7. Architecture Decision Record

This section documents the significant design choices made during development, the alternatives considered, and the reasons for each decision. These records exist so future maintainers do not undo decisions that had non-obvious reasoning.

ADR-001 Hygiene Findings Separated from Compliance Gate

Decision	Non-blocking Hygiene findings (HYG-001/002/003) do not affect ComplianceStatus. They contribute to RiskScore but cannot make an entity NonCompliant.
Problem	Initial runs showed 0% compliant distribution. Analysis revealed that HYG-002 was firing on all users due to missing AuditLog scope. Because HYG findings were gating compliance, the entire tenant appeared non-compliant from a data gap.
Alternatives considered	(1) Remove Hygiene rules from FullScan entirely. Rejected — hygiene signals are real risk. (2) Keep Hygiene in compliance gate. Rejected — operational debt should not inflate compliance statistics. (3) Separate the gate. Chosen.
Exception	HYG-004 (no MFA on privileged account) remains blocking. MFA on privileged accounts is a hard security control, not operational hygiene.
Trade-off	An entity with severe inactivity and stale password will show Compliant if it has no compliance findings. This is by design — ComplianceStatus reflects policy alignment, not operational health.

ADR-002 Fixed Theoretical Maximum for Score Normalisation

Decision	RiskScore is normalised against a fixed theoretical maximum of 900, not against the worst-scoring entity in the current run.
Problem	Tenant-relative normalisation ($\text{score} / \text{max-in-run} \times 100$) would compress scores when one severely non-compliant entity exists. An entity with score 200 raw would show 25.0 if the worst is 800, but 100.0 if the worst is 200. The same real-world state would produce different numbers depending on who else is in the tenant.
Alternatives considered	Tenant-relative normalisation (rejected — scores not comparable across runs or tenants). Uncapped raw scores (rejected — scores of 1220 have no intuitive meaning).
Chosen	Fixed cap at 900 (sum of all critical-weight rules that realistically co-occur). Scores are now comparable across tenants and over time.
Trade-off	The theoretical max is a judgment call. If future rules push realistic worst-case scores above 900, the cap should be revised.

ADR-003 Subscription-Agnostic RBAC Collection

Decision	RbacCollector enumerates all subscriptions the service account can see, not a configured subset.
Reasoning	Scoping to a known subscription list would produce false confidence — the engine would report clean where it had simply not looked. IAM risk in an unknown subscription is still IAM risk.
Opt-in restriction	SubscriptionFilter parameter allows scope restriction when genuinely needed (e.g. business unit scoping). It is opt-in, not the default.

Trade-off	Performance scales with subscription count. Each subscription is a sequential ARM call. For large organisations this is the primary performance bottleneck.

ADR-004 Declarative Suppression in Rules.json

Decision	Rule suppression relationships (suppressesOnMatch, supersededBy) are declared in Rules.json and processed by a shared Invoke-SuppressionPass function, not hardcoded in individual processors.
Problem	Early versions had suppression logic hardcoded per rule in each processor. Adding a new suppression required code changes in multiple files.
Chosen	Declare suppression in data (Rules.json). Invoke-SuppressionPass runs at the return of every orchestrator and applies all declared relationships. Adding a new suppression requires only a Rules.json change.
Exception	Scope-aware RBAC suppression (RBAC-003 suppresses RBAC-002 only for same entity+scope+role) requires entity and scope context that cannot be expressed in a simple suppressesOnMatch list. This is handled by a dedicated scope-key pass in RbacRuleProcessor before the shared pass runs.

ADR-005 IDENT-004 Removal — Single Rule for Employment Status

Decision	IDENT-004 (Unknown employment status) was removed. IDENT-003 now handles both non-Active and Unknown status via the unknownTrigger parameter.
Problem	IDENT-003 and IDENT-004 were both evaluating the same field (EmploymentStatus) for the same entity, producing two findings with combined weight 80 for a single data point. 15 entities were double-penalised.
Root cause	IDENT-003 fired when EmploymentStatus != Active. IDENT-004 fired when EmploymentStatus == Unknown. For Unknown status, both conditions were true simultaneously.
Fix	IDENT-003 gained unknownTrigger: true parameter. When Unknown, IDENT-003 emits a distinct message ("could not be determined — verify HR sync"). The continue statement prevents the generic != Active branch from also firing.

8. File Structure and Deployment

8.1 Repository Structure

```
ValidationEngine/
├── ValidationEngine.ps1      # Entry point – orchestration only
├── Rules.json               # Rule definitions + entitlement model
├── DataCollector.ps1        # (deprecated stub – replaced by collectors below)
├── RuleProcessor.ps1        # (deprecated stub – replaced by processors below)
├── RiskClassifier.ps1       # Classification engine
├── ReportGenerator.ps1      # Output layer
├── Collectors/
│   ├── IdentityCollector.ps1 # Microsoft Graph – users, groups, memberships
│   └── RbacCollector.ps1     # Azure ARM – subscriptions, role assignments
├── Rules/
│   ├── FindingSchema.ps1    # New-ComplianceFinding + Invoke-SuppressionPass
│   ├── IDRuleProcessor.ps1  # Identity, Access, Hygiene, Architecture rules
│   ├── RbacRuleProcessor.ps1 # RBAC rules
│   └── CorrelationRuleProcessor.ps1 # Cross-plane rules
├── Tests/
│   ├── Test-FindingSchema.ps1
│   ├── Test-IdentityCollector.ps1
│   ├── Test-IdentityRuleProcessor.ps1
│   ├── Test-RbacCollector.ps1
│   ├── Test-RbacRuleProcessor.ps1
│   ├── Test-CorrelationRuleProcessor.ps1
│   ├── Test-RiskClassifier.ps1
│   └── Test-ReportGenerator.ps1
└── Output/                  # Generated at runtime
    ├── FullScan_Report_*.json
    ├── FullScan_Report_*.csv
    └── LastRun_DriftState.json
```

8.2 Prerequisites

Requirement	Version / Notes
PowerShell	7.x recommended. 5.1 compatible with minor syntax adjustments.
Microsoft.Graph module	Connect-MgGraph -Scopes "User.Read.All","Group.Read.All","GroupMember.Read.All","UserAuthenticationMethod.Read.All","AuditLog.Read.All"
Az module	Az.Accounts + Az.Resources. Connect-AzAccount before running.
Entra ID P1 or P2	Required for signInActivity (LastSignInDateTime). Engine degrades gracefully without it — HYG-001/002 fire on all users.
Service account permissions	Read-only on Entra ID (above Graph scopes). Reader or higher on target Azure subscriptions.

8.3 Output Files

File	Format	Produced By	Content
FullScan_Report_{timestamp}.json	JSON	-ExportJson switch	Full EntityRiskStates array + Summary object + mode metadata
FullScan_Report_{timestamp}.csv	CSV (UTF-8)	-ExportCsv switch	24 columns, one row per entity×finding. See Section 8.4.

File	Format	Produced By	Content
LastRun_DriftState.json	JSON	-StoreDriftState switch	Compact state file. Read by next run to compute RiskTrend.
ErrorLog_{timestamp}.csv	CSV	-ErrorLogPath parameter	Non-fatal collection errors. Timestamp, Source, Detail.

8.4 CSV Column Reference (24 columns)

Column	Category	Description
EntityId	Identity	Azure AD Object ID
UPN	Identity	UserPrincipalName (or DisplayName for SP/Group)
DisplayName	Identity	Display name
EntityType	Identity	User Group ServicePrincipal
EmployeeType	Identity	Raw HR-sourced EmployeeType value from Entra ID
EmploymentStatus	Identity	Normalised: Active Terminated Offboarded Unknown Guest
IsPrivileged	Access	True if member of any privileged-tier group
Tier	Access	Highest entitlement tier across all group memberships
Groups	Access	Pipe-delimited list of group display names
ComplianceStatus	Risk	Compliant NonCompliant Blocking
RiskLevel	Risk	None Low Medium High Critical
RiskScore	Risk	Normalised 0–100
RiskScoreRaw	Risk	Raw weight sum (analytics use only)
RiskDensity	Risk	RiskScore ÷ total violation count
HygieneIssueCount	Risk	Count of non-blocking Hygiene findings
RecommendedAction	Risk	None HygieneReview Alert Review Escalate
RuleId	Finding	e.g. RBAC-003. One row per finding.
Category	Finding	Rule category
Severity	Finding	Low Medium High Critical
Weight	Finding	Rule weight (0–100)
Blocking	Finding	True False
Details	Finding	Human-readable finding description
Timestamp	Finding	ISO 8601 UTC — SIEM-ready
Mode	Finding	FullScan DriftOnly PreProvision

9. Performance Considerations

9.1 Current Architecture — Sequential Processing

The engine currently processes subscriptions sequentially in RbacCollector and evaluates rules per-entity sequentially in each processor. This is intentional for the current scale — sequential processing is easier to debug, produces deterministic output, and avoids Az module thread-safety issues.

Operation	Approximate Cost	Scales With
Graph: Get-MgUser -All	1 API call (paginated)	Total user count
Graph: Get-MgGroup -All	1 API call (paginated)	Total group count
Graph: Get-MgGroupMember per group	1 API call per group	Group count (major bottleneck for large tenants)
Graph: Get-MgUserAuthenticationMethod	1 API call per user (only with -ResolveMfa)	User count — significant latency for large tenants
ARM: Get-AzRoleAssignment per subscription	1 API call per subscription	Subscription count
Rule evaluation	In-memory — negligible	Finding count

9.2 Throttling

Microsoft Graph throttles requests at the tenant level. The engine does not implement retry logic for 429 (Too Many Requests) responses. In practice, graph throttling surfaces as a failed collection error logged to ErrorLogPath. For large tenants (10,000+ users), the -ResolveMfa switch should be used with caution or omitted from scheduled runs.

9.3 Estimated Entity Limits

NOTE These estimates are based on the current sequential architecture. They have not been validated against large production tenants. Treat them as design-time guidance, not production guarantees.

Tenant Size	Estimated FullScan Duration	Notes
< 500 users	2–5 minutes	Observed in development. -ResolveMfa adds ~30s per 100 users.
500–2000 users	5–15 minutes	Group membership calls become the primary bottleneck.
2000–10000 users	15–60 minutes	Consider DriftOnly for frequent checks. FullScan weekly.
> 10000 users	Not validated	Parallelisation of group membership calls and/or ARM calls would be required. See 9.4.

9.4 Future: Parallelisation Approach

- Group membership calls (Get-MgGroupMember) are the primary bottleneck in large tenants. These are independent per group and could be parallelised using PowerShell 7 ForEach-Object -Parallel.
- ARM subscription calls are also independent per subscription and are natural candidates for parallel execution. Az module thread safety must be validated before implementing.

- Rule evaluation is currently per-entity sequential but is pure in-memory and negligible relative to API call time.

10. Lifecycle Integration (JML)

The engine is designed as the validation layer in a broader Joiner / Mover / Leaver (JML) workflow. The integration model is defined here even though the JML orchestration layer is under active development. The engine's PreProvision mode is the primary integration point.

10.1 Joiner

Trigger	New hire record created in HR system / Entra ID provisioned
Engine call	PreProvision mode with new user's Azure AD Object ID
Decision = Pass	Proceed with group assignment and resource access provisioning
Decision = Fail	Block provisioning. Surface Reasons[] to provisioning workflow for ticket creation or human review.
Rules active	JOIN-001 (no employment record), JOIN-002 (restricted role for job title), IDENT-002 (malformed UPN), ENT-002 (employment type vs group tier)

10.2 Mover

Trigger	Department/role change event in HR system
Engine call	PreProvision mode for the new role context (engine evaluates current state — provisioner must first revoke old memberships)
Decision = Pass	Assign new role groups. Run DriftOnly after provisioning to confirm clean state.
Decision = Fail	Block role change. Surface Reasons[] — typically ENT-002 (employment type vs new role tier) or ACCESS-001 (contractor into management group).
Follow-up	FullScan within 24h of move to confirm no residual access from old role.

10.3 Leaver

Trigger	Termination event in HR system
Engine call	FullScan (or targeted DriftOnly) after account disable and group revocation
Expected result	ACCESS-002 should not fire (terminated account with group memberships). If ACCESS-002 fires, remediation is incomplete.
RBAC cleanup	CORR-002 fires if disabled account retains Azure RBAC. This is the most common post-termination finding in practice.
Verification	Run FullScan with -StoreDriftState 48h after termination. Compare drift trend to confirm clean state.

10.4 Periodic Governance Scans

Schedule	Mode	Purpose
Daily	DriftOnly	Detect structural RBAC changes between FullScan runs. Fast — same API volume, fewer rules.
Weekly	FullScan - StoreDriftState	Full compliance baseline. Drift trend computed against previous stored state.
Monthly	FullScan -ResolveMfa -ExportCsv - ExportJson	Full run with MFA resolution for audit reporting. CSV for spreadsheet consumers, JSON for downstream systems.

11. Extending the Engine — Adding a New Rule

The engine is data-driven. Adding a new rule requires three steps and no changes to the core orchestration or classification layers.

Step 1 — Define the rule in Rules.json

```
{
  "id": "CORR-006",
  "description": "Service principal holds Owner at subscription scope",
  "category": "Correlation",
  "severity": "Critical",
  "weight": 100,
  "blocking": true,
  "type": "Evaluate-CrossPlaneExposure",
  "parameters": {
    "principalType": "ServicePrincipal",
    "forbiddenRoles": ["Owner"],
    "forbiddenScopes": ["/subscriptions/"]
  },
  "suppressesOnMatch": [],
  "supersededBy": ""
}
```

Step 2 — Add an evaluator function to the relevant processor

The type field maps to a dispatch table in each processor. Add a new function and register it:

```
# In CorrelationRuleProcessor.ps1 dispatch table:
$Script:CorrelationDispatch = @{
  "Evaluate-CrossPlaneExposure" = { param($r,$id,$rbac,$idx,$rd) Evaluate-
CrossPlaneExposure $r $id $rbac $idx $rd }
  "Evaluate-ServicePrincipalExposure" = { param($r,$id,$rbac,$idx,$rd) Evaluate-
ServicePrincipalExposure $r $id $rbac $idx $rd }
}
```

Step 3 — Add a test in Tests/

Every new rule needs a test file with at minimum: (1) a positive case that confirms the rule fires, (2) a negative case that confirms it does not fire on a compliant entity, and (3) a suppression case if suppressesOnMatch is set.

INVARIANT New rules must not break the classifier invariant: an entity with ComplianceStatus=Compliant cannot have ViolationCount > 0. The classifier enforces this with an explicit throw. Any new rule that produces findings must have Blocking set to either true or false — it cannot be null.

12. Future Enhancements

Enhancement	Value	Complexity	Notes
PIM eligibility detection	High	Medium	Entra ID Privileged Identity Management assignments are not currently evaluated. PIM-eligible Owner roles are invisible to the current RBAC rules.
Conditional Access policy ingestion	High	High	Evaluate whether CA policies adequately protect privileged accounts. Would require Graph Identity.SignIn.Read.All scope.
Drift scoring over time	Medium	Low	Drift state file already stored per run. A trend aggregator that reads historical state files would produce a risk trajectory chart.
Access review correlation	Medium	Medium	Cross-reference Entra access reviews with current group memberships. Identify groups where review was completed but access not revoked.
Parallelised group membership collection	High (large tenants)	Medium	ForEach-Object -Parallel in PowerShell 7. Thread-safety validation required for Az module context.
SIEM integration	High	Low	Findings already carry ISO 8601 timestamps and stable RuleIds. A Sentinel/Splunk connector could consume the JSON output directly.
Entra ID directory role evaluation	Medium	Medium	Global Administrators and Privileged Role Administrators are currently outside RBAC scope. Directory roles require separate Graph calls.